- Define one or more *requirements* for the policy.
- Define one or more *handlers* for each requirement.
- Register the handlers in the DI container.

With the exception of the handler, these steps are identical to the declarative authorization approach with the `[Authorize]` attribute, so I run through them only briefly here.

First, you can create a simple `IAuthorizationRequirement`. As with many requirements, this contains no data and simply implements the marker interface:

```
public class IsRecipeOwnerRequirement : IAuthorizationRequirement { }
```

Defining the policy in Program.cs is similarly simple, as you have only a single requirement. Note that there's nothing resource-specific in any of this code so far:

```
builder.Services.AddAuthorization(options => {
    options.AddPolicy("CanManageRecipe", policyBuilder =>
        policyBuilder.AddRequirements(new IsRecipeOwnerRequirement()));
});
```

You're halfway there. All you need to do now is create an authorization handler for `IsRecipeOwnerRequirement` and register it with the DI container.

24.5.2 Creating a resource-based AuthorizationHandler